

Generative Adversarial Networks and Summary of Course

FYS5429/9429

Morten Hjorth-Jensen

Department of Physics & CCSE, University of Oslo

May 21, 2026

Outline I

- 1 Motivation: VAEs and Their Limits
- 2 Generative Adversarial Networks
- 3 What Is a GAN?
- 4 Mathematical Formulation
- 5 Optimal Discriminator and JS Divergence
- 6 Loss Functions in Practice
- 7 Training Challenges
- 8 Evaluating GANs
- 9 Architecture Design

Outline II

- 10 Extensions and Variants
- 11 Summary and Exercises
- 12 Comparing Generative Models: VAEs, Diffusion, and GANs
- 13 Summary of course

Generative models: the common thread

All deep generative models share one goal: learn a distribution $p_{\theta}(x)$ over data x that we can both evaluate and sample from.

Family	Latent	Training signal	Sample cost
VAE	$\mathbf{h} \in \mathbb{R}^{d_h}, d_h \ll d_x$	ELBO	1 decoder pass
Diffusion	$\mathbf{x}_t \in \mathbb{R}^{d_x}$ (full dim)	ELBO , T levels	T denoiser calls
GAN	$\mathbf{z} \in \mathbb{R}^k$	Adversarial	1 generator pass

Key insight

Diffusion models are **hierarchical VAEs** with T latent levels, a fixed (non-learned) encoder, and full-dimensional latents. Understanding VAEs is the fastest route to understanding diffusion.

Transition: From Likelihood to Adversarial Training

So far we studied models maximising (a lower bound on) $\log p(\mathbf{x})$:

Model	Objective	Key idea
VAE	Maximise ELBO	Variational inference, 1 latent level
DDPM	Maximise ELBO (T levels)	Fixed encoder, noise prediction
GAN	No likelihood bound	Adversarial game

Fundamental difference

GANs never evaluate $\log p_{\theta}(\mathbf{x})$. A **discriminator** D provides the training signal by classifying real vs. fake, driving the **generator** G to match p_r — without any likelihood computation.

Benefit: sharper samples, no ℓ_2 -blurring. *Cost:* no likelihood, mode collapse risk, instability.

Generative Adversarial Networks

A **GAN** (Goodfellow et al., NeurIPS 2014) pits two neural networks against each other in a zero-sum game:

Generator G

- Input: noise $z \sim p_z(z)$
- Output: synthetic sample $x = G(z; \theta^{(g)})$
- Goal: fool the discriminator
- Implicitly defines distribution p_g

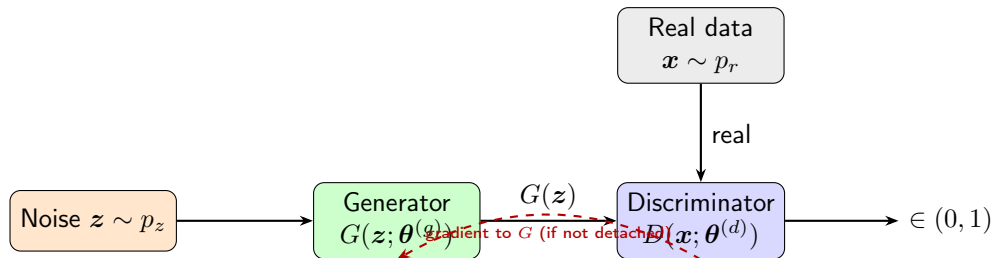
Discriminator D

- Input: image x
- Output: $D(x; \theta^{(d)}) \in (0, 1)$
 $= \Pr(x \text{ is real})$
- Goal: distinguish real from fake

Key insight

Neither network sees the other's parameters. They improve *each other* through adversarial feedback alone.

GAN Architecture (Schematic)



- G maps $z \in \mathbb{R}^k \rightarrow \mathbb{R}^d$ (noise $\rightarrow$ data space)
- D maps $x \in \mathbb{R}^d \rightarrow (0, 1)$ (image $\rightarrow$ probability)
- Training alternates: freeze G and update D ; freeze D and update G

Three learning settings:

- 1 **Unsupervised** — no labels; G learns the full data manifold
- 2 **Supervised** — conditional GAN conditions G on class label
- 3 **Semi-supervised** — D doubles as a feature extractor

Applications:

- Image synthesis / super-resolution
- Text-to-image translation
- Video prediction
- Data augmentation for rare events
- Anomaly detection
- Scientific simulation:
particle physics, drug discovery,
climate modelling

Symbol	Meaning
$p_r(\mathbf{x})$	Real data distribution (unknown; represented by training set)
$p_z(\mathbf{z})$	Latent noise prior, e.g. $\mathcal{N}(\mathbf{0}, I)$
$G(\mathbf{z}; \boldsymbol{\theta}^{(g)})$	Generator: differentiable map $\mathbb{R}^k \rightarrow \mathbb{R}^d$
p_g	Generator distribution = push-forward of p_z through G
$D(\mathbf{x}; \boldsymbol{\theta}^{(d)}) \in (0, 1)$	Discriminator: $\Pr(\mathbf{x} \text{ is real})$

Implicit distribution. For invertible G the change-of-variables gives:

$$p_g(\mathbf{x}) = p_z(G^{-1}(\mathbf{x})) \left| \det \frac{\partial G^{-1}}{\partial \mathbf{x}} \right|.$$

In practice G is not invertible; p_g is an *implicit* distribution accessed only through sampling.

The Minimax Objective

Value function (Goodfellow et al., 2014)

$$\min_G \max_D V(G, D) = \underbrace{\mathbb{E}_{\mathbf{x} \sim p_r} [\log D(\mathbf{x})]}_{D \text{ correct on real}} + \underbrace{\mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))]}_{D \text{ correct on fake}}$$

Discriminator (maximise V):

- $D(\mathbf{x}) \rightarrow 1$ on real data
- $D(G(\mathbf{z})) \rightarrow 0$ on generated data

Generator (minimise V):

- $D(G(\mathbf{z})) \rightarrow 1$ (fool D)
- $\mathbb{E}_{p_r}[\log D(\mathbf{x})]$ is constant w.r.t. G , so G minimises $\mathbb{E}[\log(1 - D(G(\mathbf{z})))]$

Non-Saturating Generator Loss

Problem with the original objective. Early in training D is strong, so $D(G(\mathbf{z})) \approx 0$:

$$\frac{\partial}{\partial \theta^{(g)}} \log(1 - D(G(\mathbf{z}))) \approx \frac{\partial}{\partial \theta^{(g)}} \log 1 = 0.$$

The gradient *vanishes* before G has learned anything.

Fix: non-saturating loss

Instead of minimising $\log(1 - D(G(\mathbf{z})))$, maximise $\log D(G(\mathbf{z}))$:

$$\mathcal{L}_G^{\text{non-sat}} = -\mathbb{E}_{\mathbf{z} \sim p_z} [\log D(G(\mathbf{z}))].$$

Same Nash equilibrium; stronger gradient when $D(G(\mathbf{z})) \approx 0$.

Loss form	Value at $D(G(\mathbf{z})) = \varepsilon \approx 0$	gradient
$\log(1 - \varepsilon)$	$\approx -\varepsilon$	≈ 1 (small)
$-\log \varepsilon$	$+\infty$	$1/\varepsilon \rightarrow \infty$ (large)

The Alternating Gradient Algorithm

One training iteration

- 1 Freeze G ; gradient ascent on D :

$$\theta^{(d)} \leftarrow \theta^{(d)} + \eta \nabla_{\theta^{(d)}} V(G, D).$$

- 2 Freeze D ; gradient descent on G :

$$\theta^{(g)} \leftarrow \theta^{(g)} - \eta \nabla_{\theta^{(g)}} \mathcal{L}_G.$$

- One D step per G step in practice ($k = 1$).
- Fake images are **detached** from the computation graph during the D step to prevent spurious gradients flowing into G .
- Both networks update simultaneously — this does *not* guarantee convergence (see Section ??).

Deriving the Optimal Discriminator

Fix G (hence fix p_g). Write V as an integral:

$$V(G, D) = \int_{\mathbf{x}} [p_r(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x}))] d\mathbf{x}.$$

For each $\mathbf{x}$ independently, maximise the integrand $f(t) = A \log t + B \log(1 - t)$, $A = p_r(\mathbf{x})$, $B = p_g(\mathbf{x})$, $t = D(\mathbf{x}) \in (0, 1)$:

$$f'(t) = \frac{A}{t} - \frac{B}{1-t} = \frac{A - (A+B)t}{t(1-t)} = 0 \implies t^* = \frac{A}{A+B}.$$

Check: $f''(t^*) = -A/(t^*)^2 - B/(1-t^*)^2 < 0$ ✓ (maximum).

Optimal discriminator

$$D^*(\mathbf{x}) = \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_g(\mathbf{x})}.$$

This is the **Bayes-optimal classifier** for the two-class problem {real, fake} under equal class priors.

Nash equilibrium. At $p_g = p_r$:

$$D^*(\mathbf{x}) = \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_r(\mathbf{x})} = \frac{1}{2}.$$

The discriminator cannot distinguish real from fake — it is reduced to random guessing.

Loss at the equilibrium. Substituting $D^* = 1/2$:

$$\begin{aligned} V(G^*, D^*) &= \log \frac{1}{2} \int p_r d\mathbf{x} + \log \frac{1}{2} \int p_r d\mathbf{x} \\ &= -2 \log 2 \approx -1.386. \end{aligned}$$

Convergence indicator

In practice, monitor $D(\mathbf{x})$ and $D(G(\mathbf{z}))$ during training. Both should converge to 0.5 at equilibrium. If $D(G(\mathbf{z})) \ll 0.5$, the generator is losing. If $D(\mathbf{x}) \ll 0.5$, the discriminator is failing.

Kullback-Leibler divergence:

$$D_{\text{KL}}(p\|q) = \int p(\mathbf{x}) \log \frac{p(\mathbf{x})}{q(\mathbf{x})} d\mathbf{x} \geq 0,$$

with equality iff $p = q$ a.e. (Gibbs' inequality). Note: $D_{\text{KL}}(p\|q) \neq D_{\text{KL}}(q\|p)$ in general.

Jensen-Shannon divergence (symmetric, bounded):

$$D_{\text{JS}}(p\|q) = \frac{1}{2}D_{\text{KL}}\left(p\left\|\frac{p+q}{2}\right.\right) + \frac{1}{2}D_{\text{KL}}\left(q\left\|\frac{p+q}{2}\right.\right).$$

Properties of D_{JS}

- Symmetric: $D_{\text{JS}}(p\|q) = D_{\text{JS}}(q\|p)$
- Non-negative: $D_{\text{JS}}(p\|q) \geq 0$, with equality iff $p = q$
- Bounded: $D_{\text{JS}}(p\|q) \in [0, \log 2]$ for any p, q
- Related to total variation: $D_{\text{JS}}(p\|q) \leq \log 2 \cdot \text{TV}(p, q)$

GAN Loss = Jensen-Shannon Divergence

Expanding $D_{\text{JS}}(p_r \| p_g)$ and simplifying:

$$\begin{aligned} D_{\text{JS}}(p_r \| p_g) &= \frac{1}{2} D_{\text{KL}}\left(p_r \left\| \frac{p_r + p_g}{2}\right.\right) + \frac{1}{2} D_{\text{KL}}\left(p_g \left\| \frac{p_r + p_g}{2}\right.\right) \\ &= \frac{1}{2} \left(\log 2 + \int p_r \log \frac{p_r}{p_r + p_g} d\mathbf{x} \right) + \frac{1}{2} \left(\log 2 + \int p_g \log \frac{p_g}{p_r + p_g} d\mathbf{x} \right). \end{aligned}$$

Recognising $V(G, D^*)$ in the integrals:

Main theorem (Goodfellow et al. 2014)

$$V(G, D^*) = 2 D_{\text{JS}}(p_r \| p_g) - 2 \log 2.$$

Conclusion. The GAN minimax objective is equivalent to minimising the Jensen-Shannon divergence between p_r and p_g . The global minimum $-2 \log 2$ is achieved when $p_g = p_r$.

Binary Cross-Entropy Losses

Discriminator loss (real label = s , fake label = 0):

$$\mathcal{L}_D = -\frac{1}{N} \sum_{i=1}^N \log D(\mathbf{x}_i) - \frac{1}{M} \sum_{j=1}^M \log(1 - D(G(\mathbf{z}_j))).$$

Generator loss (non-saturating):

$$\mathcal{L}_G = -\frac{1}{M} \sum_{j=1}^M \log D(G(\mathbf{z}_j)).$$

One-sided label smoothing ($s = 0.9$)

Replace real target 1 with $s < 1$. The optimal discriminator becomes

$D_s^*(\mathbf{x}) = s \cdot p_r(\mathbf{x}) / (p_r(\mathbf{x}) + p_g(\mathbf{x}))$, preventing overconfidence and keeping gradients to G alive.

Wasserstein Distance (WGAN)

Problem with JS divergence. When p_r and p_g have non-overlapping supports (common early in training): $D_{JS}(p_r \| p_g) = \log 2$ constantly, giving zero gradient everywhere.

Wasserstein-1 distance (*Earth-mover*):

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} \|\mathbf{x} - \mathbf{y}\|$$

$\Pi(p_r, p_g)$: all couplings with marginals p_r, p_g .

Kantorovich-Rubinstein duality

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} (\mathbb{E}_{p_r}[f] - \mathbb{E}_{p_g}[f]).$$

Sup. over all 1-Lipschitz functions f .

Advantage

W gives meaningful gradients even when $p_r \cap p_g = \emptyset$. Training is more stable.

WGAN critic. Replace D with a critic f_w (no sigmoid):

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{p_g}[f_w(\mathbf{x})] - \mathbb{E}_{p_r}[f_w(\mathbf{x})] + \lambda \mathcal{L}_{\text{GP}}.$$

Gradient penalty (Gulrajani et al., 2017):

$$\mathcal{L}_{\text{GP}} = \mathbb{E}_{\hat{\mathbf{x}}} \left[\left(\|\nabla_{\hat{\mathbf{x}}} f_w(\hat{\mathbf{x}})\|_2 - 1 \right)^2 \right],$$

where $\hat{\mathbf{x}} = \epsilon \mathbf{x} + (1 - \epsilon)G(\mathbf{z})$, $\epsilon \sim U[0, 1]$ (uniform on straight lines between real and fake points).

- Enforces the 1-Lipschitz constraint via soft penalisation.
- More stable than weight clipping (original WGAN).
- $\lambda = 10$ is a standard choice.

When does it occur? When D is perfect early in training: $D(\mathbf{x}) = 1 \ \forall \mathbf{x} \sim p_r$ and $D(G(\mathbf{z})) = 0 \ \forall \mathbf{z} \sim p_z$.

Original G gradient:

$$\nabla_{\boldsymbol{\theta}(g)} \mathbb{E}[\log(1 - D(G(\mathbf{z})))] \approx \nabla_{\boldsymbol{\theta}(g)} \mathbb{E}[\log 1] = \mathbf{0}.$$

No information flows to G .

Non-saturating loss $-\log D(G(\mathbf{z}))$:

$$\nabla_{\boldsymbol{\theta}(g)} (-\log D(G(\mathbf{z}))) = -\frac{1}{D(G(\mathbf{z}))} \nabla_{\boldsymbol{\theta}(g)} D(G(\mathbf{z})).$$

When $D(G(\mathbf{z})) \approx 0$, the factor $1/D(G(\mathbf{z})) \rightarrow \infty$ amplifies the gradient — learning continues.

Definition. The generator maps many different noise vectors z to the *same* (or very few) output(s):

$$\text{supp}(p_g) \ll \text{supp}(p_r).$$

The JS divergence can still be low on the covered modes, so the loss does not detect collapse.

Why it happens. Given the current discriminator, G finds a single “safe” output that fools D ; D then adapts, and G shifts to another single output. The two networks chase each other indefinitely.

Mitigations:

- **Minibatch discrimination:** D sees statistics of a batch, not individual images — diversity is rewarded.
- **Unrolled GANs:** G optimises against a future D , preventing short-sighted collapse.
- **Spectral normalisation:** stabilises D by bounding its Lipschitz constant.
- **WGAN:** Wasserstein distance measures full distribution mismatch, not just modes.

Non-Convergence and the Game-Theoretic View

GAN training solves a **two-player non-cooperative game**:

$$G^* = \arg \min_G \arg \max_D V(G, D).$$

Nash equilibrium exists at $p_g = p_r$, $D^* \equiv \frac{1}{2}$, but:

- Simultaneous gradient updates do *not* guarantee convergence to a Nash equilibrium.
- The loss landscape of each player changes at every step (non-stationary environment).
- If $\max_D V(G, D)$ is not convex in $\theta^{(g)}$, global convergence fails even in theory.

Practical stabilisation:

- Lower β_1 in Adam (reduces momentum: 0.5 instead of 0.9)
- One-sided label smoothing
- Spectral normalisation of D 's weights
- Progressive growing (ProGAN), self-attention (SAGAN)
- Two-time-scale updates: different learning rates for G and D

Evaluation Metrics: Overview

Evaluating generative models is hard: there is no ground truth output to compare against.

Metric		Measures	Limitation
Inception (IS)	Score	Quality + diversity of generated images	Requires pretrained classifier
FID		Distance between real/fake feature distributions	Sensitive to sample size
MMD		Kernel-based distribution distance	Choice of kernel matters
Precision/Recall		Coverage vs. fidelity separately	Requires feature extractor

Maximum Mean Discrepancy (MMD)

Definition in RKHS. Let $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ be a kernel. The **MMD** between distributions p, q is:

$$\text{MMD}^2(p, q) = \mathbb{E}_{\mathbf{x}, \mathbf{x}' \sim p}[k(\mathbf{x}, \mathbf{x}')] - 2 \mathbb{E}_{\mathbf{x} \sim p, \mathbf{y} \sim q}[k(\mathbf{x}, \mathbf{y})] + \mathbb{E}_{\mathbf{y}, \mathbf{y}' \sim q}[k(\mathbf{y}, \mathbf{y}')].$$

With the Gaussian kernel $k(\mathbf{x}, \mathbf{y}) = \exp(-\|\mathbf{x} - \mathbf{y}\|^2 / 2\sigma^2)$: MMD = 0 iff $p = q$ (characteristic kernel property).

Unbiased empirical estimator

Given n real samples $\{\mathbf{x}_i\}$ and m fake samples $\{\mathbf{y}_j\}$:

$$\widehat{\text{MMD}}^2 = \frac{1}{n(n-1)} \sum_{i \neq j} k(\mathbf{x}_i, \mathbf{x}_j) - \frac{2}{nm} \sum_{i,j} k(\mathbf{x}_i, \mathbf{y}_j) + \frac{1}{m(m-1)} \sum_{i \neq j} k(\mathbf{y}_i, \mathbf{y}_j).$$

Diagonal terms ($i = j$) excluded for unbiasedness.

Fréchet Inception Distance (FID)

FID is currently the most widely used GAN metric.

Procedure:

- 1 Extract feature vectors from the **InceptionV3** penultimate layer for n real images and n generated images.
- 2 Fit Gaussians: $\mathcal{N}(\mu_r, \Sigma_r)$ and $\mathcal{N}(\mu_g, \Sigma_g)$.
- 3 Compute the **Fréchet distance**:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right).$$

- Lower FID = better quality and diversity.
- State-of-the-art GANs achieve $\text{FID} < 5$ on MNIST; < 10 on CIFAR-10.
- Sensitive to sample size n — use $\geq 10\,000$ samples.

Fully-connected generator (Goodfellow et al., 2014 style):

$$z \in \mathbb{R}^{100} \xrightarrow{\text{Lin}+\text{BN}+\text{LReLU}} \mathbb{R}^{256} \xrightarrow{\text{Lin}+\text{BN}+\text{LReLU}} \mathbb{R}^{512} \xrightarrow{\text{Lin}+\text{BN}+\text{LReLU}} \mathbb{R}^{1024} \xrightarrow{\text{Lin}+\text{Tanh}} \mathbb{R}^{784}$$

Design choices:

- **BatchNorm** after each Linear layer: normalises activations, allows higher LR, stabilises gradient flow
- **LeakyReLU(0.2)**: prevents dying ReLU; small gradient for negative inputs
- **Tanh** output: maps to $[-1, 1]$, matching MNIST normalised to $[-1, 1]$
- **Weights**: $\mathcal{N}(0, 0.02^2)$ (DCGAN convention)

Discriminator differences:

- **No BatchNorm**: introduces minibatch dependencies, causes instability in D
- **Dropout(0.3)**: regularises, prevents D from over-fitting on real data
- **Sigmoid** output: probability in $(0, 1)$
- **Input**: flattened image $\in \mathbb{R}^{784}$

Latent Space Interpolation

A well-trained generator has a **smooth latent space**. Linear interpolation between two noise vectors:

$$z(\alpha) = (1 - \alpha) z_1 + \alpha z_2, \quad \alpha \in [0, 1]$$

should yield a smooth visual transition between the corresponding images.

- **Smooth transitions:** generator has learned a good manifold.
- **Abrupt jumps:** mode collapse or poor latent coverage.
- Provides a qualitative test of latent space geometry that FID cannot capture.

Spherical interpolation

On a high-dimensional sphere (approximately the case for $z \sim \mathcal{N}(0, I)$), *spherical* interpolation $z(\alpha) = \frac{\sin((1-\alpha)\Omega)}{\sin \Omega} z_1 + \frac{\sin(\alpha\Omega)}{\sin \Omega} z_2$, $\cos \Omega = \hat{z}_1 \cdot \hat{z}_2$, stays on the noise manifold and gives more natural transitions.

Conditional GAN (cGAN)

A **conditional GAN** (Mirza & Osindero, 2014) conditions both G and D on auxiliary information $\mathbf{y}$ (class label, image, text):

$$\min_G \max_D V(G, D) = \mathbb{E}[\log D(\mathbf{x}|\mathbf{y})] \\ + \mathbb{E}[\log(1 - D(G(\mathbf{z}|\mathbf{y})))].$$

For MNIST: $\mathbf{y}$ is the digit class (one-hot, $\mathbb{R}^{10}$).
Concatenate $\mathbf{y}$ to both G 's input $\mathbf{z}$ and D 's input $\mathbf{x}$.

cGAN enables

- Controlled generation: specify which digit to generate
- Image-to-image translation (pix2pix)
- Text-to-image synthesis
- Style transfer

Deep Convolutional GAN (DCGAN)

Radford, Metz, Chintala (ICLR 2016) replaced fully-connected layers with **strided convolutions**:

- **Generator**: fractionally strided convolutions (transposed conv) to upsample from $z \in \mathbb{R}^{100}$ to full-resolution image.
- **Discriminator**: strided convolutions to downsample.
- BatchNorm in G (all layers except output); BatchNorm in D (except input).
- ReLU in G ; LeakyReLU in D .
- No pooling layers; no fully-connected layers (except for z).

Key advance

DCGAN produced the first visually convincing high-resolution GAN outputs and established the architectural conventions still used today. The DCGAN paper's guidelines (no BN in D input layer, LReLU slope 0.2, Adam with $\beta_1 = 0.5$) remain standard practice.

Comparison of GAN Variants

Variant	Key change	Loss	Advantage
Vanilla GAN	FC networks	JS div.	Simple
DCGAN	Conv. nets	JS div.	Better images
WGAN	Wt. clipping	Wasserstein	More stable
WGAN-GP	Grad. penalty	Wasser.+GP	No clipping
cGAN	Conditioning	JS or W	Controlled
StyleGAN	Style-based	W	SOTA

Key Equations: Summary

Concept	Formula
Minimax objective	$\min_G \max_D \mathbb{E}_{p_r}[\log D(\mathbf{x})] + \mathbb{E}_{p_z}[\log(1 - D(G(\mathbf{z})))]$
Optimal discriminator	$D^*(\mathbf{x}) = p_r(\mathbf{x}) / (p_r(\mathbf{x}) + p_g(\mathbf{x}))$
Nash equilibrium	$p_g = p_r, \quad D^* \equiv 1/2$
Loss at equilibrium	$V(G^*, D^*) = -2 \log 2$
GAN loss = JS div.	$V(G, D^*) = 2D_{\text{JS}}(p_r \ p_g) - 2 \log 2$
Non-sat. generator	$\mathcal{L}_G = -\mathbb{E}_{p_z}[\log D(G(\mathbf{z}))]$
Discriminator loss	$\mathcal{L}_D = -\mathbb{E}_{p_r}[\log D(\mathbf{x})] - \mathbb{E}_{p_z}[\log(1 - D(G(\mathbf{z})))]$
Wasserstein distance	$W(p_r, p_g) = \sup_{\ f\ _L \leq 1} (\mathbb{E}_{p_r}[f] - \mathbb{E}_{p_g}[f])$
Gradient penalty	$\lambda \mathbb{E}[(\ \nabla f_w(\hat{\mathbf{x}})\ _2 - 1)^2]$
MMD	$\sqrt{\mathbb{E}_{p \times p}[k] - 2\mathbb{E}_{p \times q}[k] + \mathbb{E}_{q \times q}[k]}$
FID	$\ \mu_r - \mu_g\ ^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$

- ❶ **Optimal discriminator.** Derive $D^*(\mathbf{x}) = p_r/(p_r + p_g)$ by differentiating $f(t) = A \log t + B \log(1 - t)$ and verify it is a maximum, not a minimum.
- ❷ **JS divergence bounds.** Prove $D_{\text{JS}}(p||q) \in [0, \log 2]$ for any two distributions and find the conditions for equality on each side.
- ❸ **Vanishing gradient.** At $D(G(\mathbf{z})) = \varepsilon \ll 1$, compute $|\nabla_{\theta(g)} \mathcal{L}_G^{\text{orig}}|$ and $|\nabla_{\theta(g)} \mathcal{L}_G^{\text{non-sat}}|$ and compare their magnitudes as $\varepsilon \rightarrow 0$.
- ❹ **Label smoothing.** Derive the optimal discriminator $D_s^*(\mathbf{x})$ when real targets are smoothed to $s \in (0, 1)$ instead of 1. Show it converges to D^* as $s \rightarrow 1$.
- ❺ **Wasserstein distance.** Compute $W(\mathcal{N}(0, 1), \mathcal{N}(\mu, 1))$ analytically and verify that $W \rightarrow 0$ as $\mu \rightarrow 0$, unlike D_{JS} when the two Gaussians have non-overlapping support.
- ❻ **Conditional GAN.** Extend the minimax objective to the conditional setting. What changes in the derivation of $D^*(\mathbf{x}|\mathbf{y})$?

Three Families: A Unified Mathematical View

All three learn $p_{\theta}(x)$ but with different objectives and inductive biases.

Criterion	VAE	Diffusion	GAN
Objective	ELBO	ELBO (T terms)	Minimax / JS
Likelihood	✓	✓	×
Latent space	Compact h	Full-dim x_t	Noise z
Encoder	Learned	Fixed schedule	None
Sampling cost	1 decoder pass	T steps	1 pass
Stability	High	High	Low–Moderate
Sample quality	Moderate	State of the art	High (sharp)
Mode coverage	Good	Excellent	Partial
Anomaly detect.	✓	✓	×

Why VAEs Produce Blurry Samples

Mathematical reason. The VAE decoder minimises:

$$-\mathbb{E}_{q_{\phi}(\mathbf{h}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{h})] \propto \mathbb{E}_{q_{\phi}}[\|\mathbf{x} - \hat{\mathbf{x}}_{\theta}(\mathbf{h})\|^2].$$

Because the posterior $q_{\phi}(\mathbf{h}|\mathbf{x})$ has nonzero variance, the decoder learns the **posterior mean over all likely completions**, which is a blur over the data manifold.

Contrast with the other two models:

Diffusion avoids blurring by

predicting only the *noise added at one step* t , not the full image. Each reverse step makes a small, committed correction. The iterative nature removes the need to average over the full distribution of plausible outputs.

GANs avoid blurring by

replacing the fixed ℓ_2 loss with an *adaptive, learned* discriminator loss. The discriminator penalises blurriness and any systematic deviation from real data, providing a richer training signal than any fixed pixel-wise objective.

The Likelihood Problem in GANs

A fundamental asymmetry. VAEs and diffusion models optimise a lower bound on $\log p(\mathbf{x})$, so they can both *generate* and *evaluate* density. GANs have **no access to** $p_g(\mathbf{x})$.

Consequences of no likelihood:

- Cannot compute log-likelihood scores for anomaly detection.
- Cannot evaluate model fit on held-out data in a principled way.
- Evaluation requires proxy metrics (FID, IS, MMD) rather than test-set log-likelihood.
- Cannot be embedded as a prior in a Bayesian framework without approximation.

Why this is a price worth paying (sometimes). The discriminator provides a *perceptual* loss that no fixed p_θ objective can replicate: it adapts to what “looks real” to a neural network, capturing texture, sharpness, and high-frequency detail that ℓ_2 and even ELBO objectives systematically underweight.

Training Stability: A Comparison

VAE — Stable

ELBO is a well-defined scalar. Both networks trained with standard SGD. KL prevents collapse when weighted correctly. Main challenge: choosing β for β -VAE.

Diffusion — Stable

MSE on noise prediction is smooth. No adversarial dynamics. Low MC gradient variance (x_t sampled in one step). Main challenge: noise schedule design.

GAN — Unstable

Non-stationary minimax landscape. No Nash convergence guarantee. $V(G, D)$ is concave-convex; simultaneous gradient ascent-descent diverges even for bilinear games. Requires WGAN-GP, spectral norm, label smoothing.

Key insight

GAN instability is *fundamental*: the minimax saddle structure prevents convergence guarantees that both VAE and DDPM enjoy.

Mode Collapse vs. Posterior Averaging: Two Failure Modes

The two pathological failure modes of generative models lie at opposite ends:

Posterior averaging (VAE)

Too much spread. The decoder must explain all data through a single mean. Result: generated images are averages over the data manifold — blurry. The model *covers* all modes but fails to commit to any one of them.

$$\hat{x} = \mathbb{E}_{q_{\phi}(h|x)}[\hat{x}_{\theta}(h)] \approx \text{mean of modes}$$

Mode collapse (GAN)

Too little spread. The generator finds a single safe output that fools D and ignores the rest of p_r . Result: sharp images but only from a few modes. $\text{supp}(p_g) \ll \text{supp}(p_r)$.

$$G(z) \approx x^* \forall z \Rightarrow D_{\text{JS}}(p_r \| p_g) \approx \log 2 \text{ (flat)}$$

Diffusion avoids both: the fixed encoder $q(x_t|x_{t-1})$ prevents collapse (no network can “cheat” the noise schedule), and the iterative denoising avoids averaging because each step commits to only a small noise correction rather than a full image prediction.

Pros

- **Likelihood bound:** ELBO usable for anomaly detection.
- **Compact latent:** $\mathbf{h}$ supports interpolation, clustering, disentanglement, property prediction.
- **Fast generation:** single decoder pass.
- **Stable training:** well-defined ELBO, no adversary.
- **Flexible:** β -VAE for disentanglement; hierarchical VAEs for expressivity.
- **Scientific use:** ideal for molecules, spectra, simulations.

Cons

- **Blurry samples:** posterior averaging is fundamental, not a capacity issue.
- **Posterior collapse:** decoder ignores $\mathbf{h}$; $\text{KL} \rightarrow 0$, latent useless.
- **ELBO gap:**
 $\log p(\mathbf{x}) = \text{ELBO} + D_{\text{KL}}(q_{\phi} \| p(\mathbf{h}|\mathbf{x})) > \text{ELBO}.$
- **Mean-field:** factorised Gaussian misses correlated posteriors.
- **Fixed-dim bottleneck:** wrong d_h hurts quality.

Detailed Pros and Cons: Diffusion Models

Pros

- **Best sample quality:** sharp, diverse, no blurring.
- **Full coverage:** fixed forward process prevents mode collapse.
- **Stable training:** MSE on noise, no adversarial dynamics.
- **Likelihood bound:** $\text{ELBO} \rightarrow \log p(x)$ as $T \rightarrow \infty$.
- **Flexible conditioning:** text, class, depth via classifier-free guidance and cross-attention.
- **Rich theory:** SDE/score connection (Song et al. 2021).

Cons

- **Slow sampling:** $T=100\text{--}1000$ steps (DDIM/DPM-Solver: 10–50).
- **No compact latent:** full-dim x_t precludes clustering or property prediction.
- **Compute-intensive:** large T , high-dim x need significant GPU memory.
- **Poor for repr. learning:** no encoder for features.
- **Schedule-sensitive:** cosine preferred over linear at high resolution.

Pros

- **Sharpest samples:** adversarial loss catches blurriness that ℓ_2 /ELBO miss.
- **Fastest inference:** single generator pass.
- **Flexible loss:** works when likelihood is intractable.
- **Rich ecosystem:** cGAN, DCGAN, WGAN-GP, StyleGAN, pix2pix.
- **Image translation:** handles paired and unpaired transfer.

Cons

- **No likelihood:** no anomaly detection or density estimation.
- **Mode collapse:** G covers only part of p_r ; FID/IS may not detect it.
- **Instability:** non-stationary minimax; no convergence guarantee; hyperparameter-sensitive.
- **JS flatness:** $D_{JS} = \log 2$ when supports don't overlap (WGAN fixes this).
- **Wasted discriminator:** D discarded after training.

When to Use Each Model

Use case	Model	Reason
Image / audio / video synthesis	Diffusion	Best quality, no collapse
Text-to-image (Stable Diffusion)	LDM	VAE compression + diffusion
Image-to-image translation	GAN	Adversarial perceptual loss
Real-time generation	VAE or GAN	Single-pass inference
Anomaly detection	VAE or Diffusion	Likelihood bound available
Representation / clustering	VAE	Compact structured latent
Scientific data (molecules)	VAE	Low-dim latent for prediction
Data augmentation	GAN or Diffusion	Sharp, diverse samples
Inpainting / super-resolution	Diffusion	Score-based posterior
Very limited data	VAE	Stable; KL regularisation

Key Equations: Complete Summary Across All Three Models

Concept	VAE / Diffusion	GAN
Encoder	$q_{\phi}(\mathbf{h} \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}, \boldsymbol{\sigma}_{\phi}^2 \mathbf{I})$ $q(\mathbf{x}_t \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\mathbf{x}_{t-1}, (1 - \alpha_t)\mathbf{I})$	None (implicit via G)
Marginal / forward	$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}$	$\mathbf{x} = G(\mathbf{z}; \boldsymbol{\theta}^{(g)})$
Objective	$\mathcal{L} = \mathbb{E}[\log p_{\theta}(\mathbf{x} \mathbf{h})] - D_{\text{KL}}(q\ p)$	$\min_G \max_D \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$
Optimal solution	$\text{DDPM: } \boldsymbol{\mu}_{\theta} = \frac{1}{\sqrt{\alpha_t}}(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\boldsymbol{\epsilon}_{\theta})$	$D^*(\mathbf{x}) = \frac{p_r(\mathbf{x})}{p_r(\mathbf{x}) + p_g(\mathbf{x})}$
Loss at equilibrium	VAE ELBO $\rightarrow \log p(\mathbf{x})$	$V(G^*, D^*) = -2 \log 2$
Divergence minimised	$D_{\text{KL}}(q\ p)$ at each level	$D_{\text{JS}}(p_r\ p_g)$
Score connection	$\mathbf{s}_{\theta} = -\boldsymbol{\epsilon}_{\theta} / \sqrt{1 - \bar{\alpha}_t}$	$D^* \rightarrow$ implicit score estimator
Sampling	1 pass (VAE); T steps (DDPM)	1 pass

Exercises: Cross-Model Comparisons

Exercises 1–3

- 1 **Common ELBO.** Show VAE (1 level) and DDPM (T levels) are special cases of the MHVAE ELBO. Identify encoder, decoder, prior in each.
- 2 **Blurriness from ℓ_2 .** For $p_r = \frac{1}{2}\delta(x-a) + \frac{1}{2}\delta(x-b)$, show the VAE decoder learns $\hat{x} = (a+b)/2$ while a GAN can produce both modes.
- 3 **Convergence.** State which property of the DDPM ELBO makes it amenable to SGD while the GAN minimax saddle is not.

Exercises 4–6

- 4 **Wasserstein vs. JS.** Compute $D_{\text{JS}}(\mathcal{N}(0, 1) \parallel \mathcal{N}(\mu, 1))$ and $W(\mathcal{N}(0, 1), \mathcal{N}(\mu, 1))$. Show D_{JS} saturates at $\log 2$ while $W = |\mu|$ grows.
- 5 **Score matching bridge.** Using $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0) = -\epsilon / \sqrt{1 - \bar{\alpha}_t}$, show DDPM equals denoising score matching (Vincent 2011).
- 6 **Latent Diffusion.** Express the LDM objective as a composition of VAE ELBO and DDPM ELBO. Identify which parameters are trained in each stage.

Diffusion models

- ① Kingma & Welling, *Auto-Encoding Variational Bayes*, ICLR 2014. 1312.6114
- ② Ho, Jain & Abbeel, *DDPM*, NeurIPS 2020. 2006.11239
- ③ Song et al., *Score-Based Generative Modeling through SDEs*, ICLR 2021. 2011.13456
- ④ Song, Meng & Ermon, *DDIM*, ICLR 2021. 2010.02502
- ⑤ Kingma et al., *Variational Diffusion Models*, NeurIPS 2021. 2107.00630
- ⑥ Nichol & Dhariwal, *Improved DDPM*, ICML 2021. 2102.09672
- ⑦ Rombach et al., *Latent Diffusion Models*, CVPR 2022. 2112.10752
- ⑧ Luo, *Understanding Diffusion Models*, 2022. 2208.11970

Generative Adversarial Networks

- ① Goodfellow et al., *Generative Adversarial Nets*, NeurIPS 2014. 1406.2661
- ② Mirza & Osindero, *Conditional GAN*, 2014. 1411.1784
- ③ Radford, Metz & Chintala, *DCGAN*, ICLR 2016. 1511.06434
- ④ Arjovsky, Chintala & Bottou, *WGAN*, ICML 2017. 1701.07875
- ⑤ Gulrajani et al., *WGAN-GP*, NeurIPS 2017. 1704.00028
- ⑥ Heusel et al., *FID*, NeurIPS 2017. 1706.08500
- ⑦ Goodfellow, Bengio & Courville, *Deep Learning*, MIT Press 2016, Ch. 20.
- ⑧ Weng, *From GAN to WGAN*, blog post, 2017.

Two-Part Structure

① Part I: Discriminative Models

Jan–Mar: NNs, CNNs, RNNs, autoencoders, transformers

② Part II: Generative Models

Apr–May: Boltzmann machines, VAEs, diffusion, GANs

Methods Covered

Discriminative

- Feed-forward NNs
- PINNs
- CNNs
- RNNs / LSTMs
- Autoencoders
- Transformers

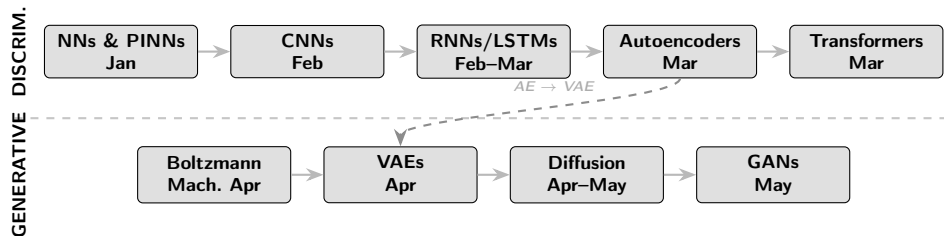
Generative

- Boltzmann machines
- VAEs
- Diffusion models
- Autoregressive
- GANs

Unifying Thread

Deep learning mathematics applied directly to problems in the physical sciences.

Course Road Map



Part I

Discriminative Deep Learning

January – March

Week 1 (Jan 19–23) — Course Introduction & NN Basics

Mathematics of Deep Learning

- Neurons, layers, activation functions
- Forward pass (compact form):
$$\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$
- Loss functions and optimisation
- Backpropagation: chain rule
- Writing a NN from scratch

Overview & Projects

- Course structure and themes
- Deep learning in the physical sciences: survey
- Discussion of project topics
- Tools: PyTorch / JAX

Goal

Build a NN from first principles before using high-level frameworks.

Week 2 (Jan 26–30) — NNs for Differential Equations

Deep Learning Mathematics (cont.)

- Optimisers: SGD, Adam, RMSProp
- Regularisation: dropout, batch norm
- Universal approximation theorem
- Practical NN coding session

Solving ODEs / PDEs

- NNs as mesh-free function approximators
- Loss encodes boundary conditions
- Advantages and limitations
- Code examples: simple ODEs

Bridge to the Physical Sciences

NNs that solve differential equations are one of the most direct applications of deep learning to the physical sciences.

Week 3 (Feb 2–6) — Physics-Informed Neural Networks (PINNs)

PINNs: Key Idea

Embed the PDE directly into the loss function:

$$\mathcal{L} = \mathcal{L}_{\text{PDE}} + \lambda \mathcal{L}_{\text{BC}}$$

- Automatic differentiation for residuals
- Hard vs. soft BC enforcement
- Schrödinger, heat, Poisson as examples

Implementation & Results

- Network architecture choices
- Collocation point sampling
- Training strategies
- Benchmarking vs. classical solvers
- Limitations of PINNs

Weeks 4–5 (Feb 9–20) — Convolutional Neural Networks

Week 4: CNN Mathematics

- Discrete convolution and cross-correlation
- Filters, feature maps, receptive fields
- Pooling: max, average, global
- Parameter sharing, equivariance
- Backprop through conv layers

Week 5: Architectures

- Classic architectures: LeNet, ResNet, U-Net
- Skip connections and residual learning
- CNNs for physics: images, fields, spectra

Applications

CNNs excel at identifying spatial patterns in simulation output, detector images, and lattice field configurations.

Recurrent Neural Networks

- Sequential data: time series, trajectories
- Vanilla RNN hidden-state update:
$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b})$$
- Backpropagation through time (BPTT)
- Vanishing / exploding gradients

Long Short-Term Memory

- LSTM gates: forget, input, output
- Cell state as long-range memory
- GRUs as a lighter alternative
- Applications:
molecular dynamics,
time-series forecasting

Weeks 8–9 (Mar 9–20) — Autoencoders & Dimensionality Reduction

Autoencoders

Encoder–decoder: $x \rightarrow z \rightarrow \hat{x}$

- Bottleneck latent space z
- Reconstruction loss $\|x - \hat{x}\|^2$
- Denoising autoencoders
- Convolutional AE architectures

Links to PCA & Implementations

- Linear AE $\equiv$ PCA (exactly)
- Kernel PCA, nonlinear extensions
- Latent space visualisation
- Applications: order parameters, phase classification, compression

The Transformer Architecture

Self-attention:

$$\text{Att}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$$

- Multi-head attention
- Positional encoding
- Encoder / decoder stacks
- Applications: quantum chemistry, PDEs and many other

Part I Summary

Five discriminative families:

- 1 Feed-forward NNs & PINNs
- 2 CNNs
- 3 RNNs / LSTMs
- 4 Autoencoders
- 5 Transformers

Each tackles a distinct structural challenge in data from the physical sciences.

Part II

Generative Deep Learning

April – May

Week 11 (Apr 6–10) — Generative Models & Boltzmann Machines

What Are Generative Models?

- Goal: learn $p_{\theta}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$
- Explicit vs. implicit density estimation
- Energy-based models: $p_{\theta}(\mathbf{x}) = e^{-E_{\theta}(\mathbf{x})} / Z(\theta)$
- Markov Chain Monte Carlo recap
- Gibbs sampling

Boltzmann Machines

- Restricted BM: visible & hidden units
- Contrastive Divergence (CD- k)
- Persistent CD and variants
- Deep Boltzmann Machines
- Statistical mechanics analogy

Week 12 (Apr 13–17) — Energy-Based Models

Energy-Based Models (Deeper Dive)

- Score matching and noise contrastive estimation
- Contrastive Divergence revisited
- Expressivity of energy functions
- Links to partition functions in statistical physics
- Training instabilities and remedies

Boltzmann Machine Codes

- Implementing an RBM from scratch
- Sampling via Gibbs chains
- Visualising learned features
- Wave-function ansatz application
- Benchmarks and diagnostics

Highlight

RBM's have been used as variational ansätze for quantum many-body ground states — a direct link to quantum computing.

VAE: Probabilistic Autoencoders

- Latent variable model:
$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z}) \mathrm{d}\mathbf{z}$$
- ELBO objective:
$$\mathcal{L} = \mathbb{E}_q[\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q||p)$$
- Reparameterisation trick
- Encoder $\rightarrow (\boldsymbol{\mu}, \boldsymbol{\sigma})$; Decoder $\rightarrow \hat{\mathbf{x}}$

VAE vs. Standard AE

- Structured, continuous latent space
- Sample: $\mathbf{z} \sim \mathcal{N}(0, I)$
- Disentanglement (β -VAE)
- Applications:
molecular generation,
anomaly detection,
data interpolation

Denoising Diffusion Probabilistic Models

- Forward: add Gaussian noise gradually
 $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$
- Reverse: learn to denoise step-by-step
- Score matching and Langevin dynamics
- Noise schedules: linear, cosine
- Link to stochastic differential equations

Why Diffusion Models?

- State-of-the-art sample quality
- Stable training vs. GANs
- Conditional generation
- Applications:
molecular structure,
protein folding,
lattice field theory

Week 15 (May 4–8) — Diffusion Models (cont.) & GANs

Diffusion: Codes & Implementations

- Implementing DDPM from scratch
- U-Net backbone for the denoiser
- Training loop and ELBO loss
- Sampling procedures
- Classifier-free guidance

Generative Adversarial Networks

Minimax:

$$\min_G \max_D \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$$

- Training dynamics and mode collapse
- Wasserstein GAN (WGAN)
- Conditional GANs (cGAN)

Advanced GAN Topics

- Architectures: DCGAN, StyleGAN, CycleGAN
- Evaluation: FID, IS, precision/recall
- GANs: fast simulation, inverse problems, lattice QCD
- GANs vs. diffusion: trade-offs

Project Presentations

Applications across the curriculum:

- PINNs for PDEs
- CNNs for detector data
- RBMs as wave-function ansätze
- VAEs for molecular design
- Diffusion / GANs for fast simulation

Weekly Road Map — Part I: Discriminative Methods

Week	Dates	Part	Topics
1	Jan 19–23	Discriminative	Course intro, NN basics, backpropagation, project discussion
2	Jan 26–30	Discriminative	NN, optimisers, regularisation, solving ODEs with NNs
3	Feb 2–6	Discriminative	Physics-Informed Neural Networks (PINNs)
4–5	Feb 9–20	Discriminative	Convolutional NNs: mathematics, architectures
6–7	Feb 23–Mar 6	Discriminative	Recurrent NNs, LSTMs, backprop through time
8–9	Mar 9–20	Discriminative	Autoencoders, PCA, latent space, implementations
10	Mar 23–27	Discriminative	Transformers, self-attention, Part I summary

Weekly Road Map — Part II: Generative Methods

Week	Dates	Part	Topics
11	Apr 6–10	Generative	Generative models overview, Boltzmann machines, MCMC, Gibbs
12	Apr 13–17	Generative	Energy-based models, wave-function ansatz
13	Apr 20–24	Generative	Variational autoencoders (VAEs), ELBO, reparameterisation
14	Apr 27–May 1	Generative	Diffusion models: forward/reverse process, score matching
15	May 4–8	Generative	Diffusion codes (DDPM), GANs: minimax, WGAN, cGAN
16	May 18–22	Generative	Advanced GANs, evaluation metrics, project presentations

What You Will Know at the End

Discriminative Skills

- Derive and implement backpropagation
- Build NNs, CNNs, RNNs, and PINNs
- Solve ODEs and PDEs with neural networks
- Design autoencoders for data reduction
- Understand attention and transformers

Generative Skills

- Train Boltzmann machines and sample from them
- Derive and implement the VAE ELBO
- Implement denoising diffusion models
- Train and stabilise GANs
- Apply generative models to problems in the physical sciences

Overarching Goal

Build both the **mathematical foundations** and the **practical implementation skills** to apply state-of-the-art deep learning methods to open problems in the physical sciences.

Perspectives

Future Directions in Advanced ML

For the Physical Sciences and Beyond

Physics

- **Many-body quantum systems:** neural-network quantum states (NQS) as variational ansätze for ground and excited states
- **High-energy physics:** jet tagging, event generation, fast detector simulation with GANs and diffusion models
- **Astro- and cosmological data:** field-level inference, gravitational-wave denoising, CMB analysis with score-based generative models
- **Condensed matter:** phase classification

Chemistry

- **Molecular generation:** VAEs, diffusion models, and flow-matching for de-novo drug and catalyst design
- **Property prediction:** graph neural networks (GNNs) on molecular graphs: energies, spectra, reaction rates
- **Quantum chemistry:** ML surrogates for DFT and CCSD(T); PINNs for the electronic Schrödinger equation
- **Reaction pathways:** transformers for retrosynthesis and transition-state prediction

Mathematics

- **Operator learning:** DeepONet and Fourier Neural Operators for mesh-free, resolution-independent PDE solvers
- **Inverse problems:** posterior sampling with score-based diffusion; learned regularisers replacing hand-crafted priors
- **Symbolic regression:** discovering equations and conservation laws directly from data
- **Uncertainty quantification:** Bayesian deep learning and conformal prediction for rigorous error bounds

Geoscience

- **Weather and climate:** data-driven emulators (GraphCast, FourCastNet) for fast global forecasts; hybrid physics–ML climate models
- **Seismology:** CNNs and RNNs for earthquake detection, waveform inversion, and induced-seismicity monitoring
- **Remote sensing:** multi-modal transformers for land use, ice-sheet dynamics, and ocean surface reconstruction
- **Subsurface imaging:** generative

Structural and Molecular Biology

- **Protein structure prediction:** AlphaFold2 and successors combine transformers with equivariant GNNs; diffusion models (RFDiffusion) for *de-novo* protein design
- **RNA and DNA:** sequence-to-function models, splicing prediction, epigenomics
- **Drug discovery:** generative molecular design, binding affinity prediction, multi-target optimisation

Systems and Medical Biology

- **Single-cell genomics:** VAEs and diffusion for cell-type discovery and trajectory inference
- **Medical imaging:** segmentation and anomaly detection with CNNs and score-based models; synthetic data for rare-disease augmentation
- **Biophysical simulations:** ML force fields (ANI, NequIP) and coarse-grained MD for proteins and membranes

Reinforcement Learning for the Physical Sciences

- **Quantum control:** RL agents learn pulse sequences for high-fidelity gate synthesis and robust quantum error correction
- **Experiment design:** active learning with RL for adaptive data acquisition in particle physics and materials science
- **Molecular design:** RL-driven exploration of chemical space guided by property reward functions
- **Plasma and fusion:** real-time magnetic confinement via deep RL policies (DeepMind / ITER)

Agentic AI in Science

- **Autonomous research loops:** LLM-based agents that formulate hypotheses, run simulations, interpret results, and iterate without human intervention
- **Multi-agent systems:** specialised agents for literature search, code generation, data analysis, and experimental planning collaborate in a shared scientific workflow
- **Human-in-the-loop science:** agentic pipelines as co-pilots for accelerating discovery in drug

Graph Neural Networks

- Natural representation of molecules, crystals, interaction networks, and complex systems
- **Equivariant GNNs**
(SE(3)-Transformers, NequIP, MACE)
enforce physical symmetries exactly
- Applications: molecular force fields, particle-physics event reconstruction, social and biological networks
- **Open challenge:** scaling equivariant architectures to very large systems efficiently

Foundation Models and Transfer Learning

- Large pre-trained models (ESM-2, ChemBERTa, ClimaX, AstroCLIP) transfer to downstream science tasks with little labelled data
- **Scientific LLMs:** models fine-tuned on papers, code, and experimental data for literature synthesis and hypothesis generation
- **Multi-modal learning:** joint text, structure, spectrum, and image representations for universal scientific models